PAWARDS MEDCORE — Módulo Pharmacy Rescue, Agente Claude Code e Integração Trello

Documento técnico-estratégico de execução direta — Dr. Replit lê, não pergunta, executa.

Este documento entrega a arquitetura completa dos 4 módulos destrancados nesta sessão: resgate de receitas não fechadas ("dinheiro na mesa"), agente conversacional Claude Code com árvore de decisão tranquila, integração Trello bidirecional LGPD-safe e painel preditivo de conversão. Seis tabelas novas, aditivas e idempotentes, oito endpoints TypeScript prontos, YAML do agente, wireframes TDAH/TOC-friendly, oito kaizens pré-autorizados e manifesto final de execução. O objetivo é transformar as receitas que hoje morrem silenciosamente em pipeline ativo de recuperação, sob autonomia guiada e blindagem LGPD, sem o paciente perceber que está sendo auditado.

A oportunidade é grande e quantificável: com **8.725 receitas já reconciliadas** no sistema atual e taxa de conversão farmacêutica tipicamente abaixo de 70% no segmento, mesmo uma recuperação parcial do gap redesenha o fluxo de caixa da clínica. O trigger SQL de comissão já está ativo (migration 027) e o painel CEO já é funcional — esses quatro módulos se encaixam como camada superior, consumindo o que já existe e adicionando inteligência de conversão no topo. Tudo multi-tenant, `clinica_id` sempre respeitado, rodapé PAWARDS®/PADCON® obrigatório.

Seção 1 — Arquitetura de dados: seis tabelas aditivas e idempotentes

Regra de ferro: todo DDL abaixo é `CREATE TABLE IF NOT EXISTS`, todo índice é `CREATE INDEX IF NOT EXISTS`, todo enum usa `DO $$ ... EXCEPTION WHEN duplicate_object`. **Zero `drizzle-kit push`**. Aplicação via `psql -f migrations/028_pharmacy_rescue.sql`. Roda 2x, 10x, 100x — resultado idêntico.

```
-- ======================================================================
-- MIGRATION 028 — PHARMACY RESCUE + AGENT + TRELLO + QUALITY SCORE
-- Aditiva. Idempotente. Multi-tenant. LGPD-aware.
-- ======================================================================
```

```
BEGIN;

-- Enums ---------------------------------------------------------------
DO $$ BEGIN
  CREATE TYPE followup_status AS ENUM (
    'enviada','aguardando_contato','cotando','fechada','sem_contato',
    'redistribuida','escalada_humano','cancelada'
  );
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

DO $$ BEGIN
  CREATE TYPE contact_channel AS ENUM ('whatsapp','telefone','email','sms','prese
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

DO $$ BEGIN
  CREATE TYPE contact_result AS ENUM (
    'sem_resposta','respondeu','numero_invalido','pediu_retorno','fechou','recuso
  );
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

DO $$ BEGIN
  CREATE TYPE agent_intent AS ENUM (
    'indefinido','cotando','nao_recebeu_orcamento','motivo_financeiro',
    'motivo_tempo','nao_quer_mais','ja_fechou_outra','escalar_humano'
  );
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

DO $$ BEGIN
  CREATE TYPE payment_gateway_kind AS ENUM ('asaas','stripe','pagarme','manual');
EXCEPTION WHEN duplicate_object THEN NULL; END $$;

-- 1) pharmacy_quote_followups ----------------------------------------
-- Rastreia cada receita emitida e o ciclo de contato com o paciente.
CREATE TABLE IF NOT EXISTS pharmacy_quote_followups (
  id                      BIGSERIAL PRIMARY KEY,
  clinica_id              INTEGER  NOT NULL,
  receita_id              INTEGER  NOT NULL,
  farmacia_original_id    INTEGER  NOT NULL,
  paciente_id             INTEGER  NOT NULL,
  valor_estimado_centavos BIGINT   NOT NULL DEFAULT 0,
  status                  followup_status NOT NULL DEFAULT 'enviada',
  tentativas              JSONB    NOT NULL DEFAULT '[]'::jsonb,
  -- tentativas: [{data, canal, resultado, observacao, operador}]
  liberado_redistribuir_em TIMESTAMPTZ,
  prazo_redistribuicao_horas INTEGER NOT NULL DEFAULT 48,
  emitida_em              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  fechada_em              TIMESTAMPTZ,
```

```
  ultima_atualizacao_em   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  metadata                JSONB    NOT NULL DEFAULT '{}'::jsonb,
  CONSTRAINT uq_followup_receita UNIQUE (clinica_id, receita_id)
);
CREATE INDEX IF NOT EXISTS ix_followup_status
  ON pharmacy_quote_followups (clinica_id, status);
CREATE INDEX IF NOT EXISTS ix_followup_liberado
  ON pharmacy_quote_followups (clinica_id, liberado_redistribuir_em)
  WHERE status IN ('enviada','aguardando_contato','sem_contato');
CREATE INDEX IF NOT EXISTS ix_followup_farmacia
  ON pharmacy_quote_followups (clinica_id, farmacia_original_id, status);

-- 2) pharmacy_redistribution ----------------------------------------
CREATE TABLE IF NOT EXISTS pharmacy_redistribution (
  id                     BIGSERIAL PRIMARY KEY,
  clinica_id             INTEGER NOT NULL,
  followup_id            BIGINT  NOT NULL REFERENCES pharmacy_quote_followups(id)
  receita_id             INTEGER NOT NULL,
  farmacias_destino      INTEGER[] NOT NULL,
  motivo                 TEXT NOT NULL,
  mensagem_paciente      TEXT,
  resposta_paciente      TEXT,
  resposta_intent        agent_intent DEFAULT 'indefinido',
  autorizado_por_master  BOOLEAN NOT NULL DEFAULT FALSE,
  master_user_id         INTEGER,
  status_cada_farmacia   JSONB NOT NULL DEFAULT '{}'::jsonb,
  -- { "12": {"status":"enviada","enviada_em":"...","resposta":"..."} }
  criada_em              TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  finalizada_em          TIMESTAMPTZ
);
CREATE INDEX IF NOT EXISTS ix_redist_followup
  ON pharmacy_redistribution (followup_id);
CREATE INDEX IF NOT EXISTS ix_redist_clinica_data
  ON pharmacy_redistribution (clinica_id, criada_em DESC);

-- 3) agent_conversation_context -------------------------------------
CREATE TABLE IF NOT EXISTS agent_conversation_context (
  id                  BIGSERIAL PRIMARY KEY,
  clinica_id          INTEGER NOT NULL,
  followup_id         BIGINT  REFERENCES pharmacy_quote_followups(id) ON DELETE S
  paciente_id         INTEGER NOT NULL,
  thread_id           TEXT NOT NULL,
  turno               INTEGER NOT NULL DEFAULT 0,
  mensagem_agente     TEXT,
  resposta_paciente   TEXT,
  intent_detectado    agent_intent NOT NULL DEFAULT 'indefinido',
  confidence          NUMERIC(4,3) NOT NULL DEFAULT 0,
```

```
  decisao_proximo     TEXT,
  escalado_humano     BOOLEAN NOT NULL DEFAULT FALSE,
  motivo_escalacao    TEXT,
  tokens_input        INTEGER DEFAULT 0,
  tokens_output       INTEGER DEFAULT 0,
  model_version       TEXT DEFAULT 'claude-sonnet-4',
  created_at          TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT uq_agent_turno UNIQUE (thread_id, turno)
);
CREATE INDEX IF NOT EXISTS ix_agent_thread
  ON agent_conversation_context (thread_id, turno);
CREATE INDEX IF NOT EXISTS ix_agent_paciente
  ON agent_conversation_context (clinica_id, paciente_id, created_at DESC);
CREATE INDEX IF NOT EXISTS ix_agent_escalado
  ON agent_conversation_context (clinica_id, escalado_humano)
  WHERE escalado_humano = TRUE;

-- 4) trello_cards_sync ------------------------------------------------
CREATE TABLE IF NOT EXISTS trello_cards_sync (
  id                   BIGSERIAL PRIMARY KEY,
  clinica_id           INTEGER NOT NULL,
  receita_id           INTEGER NOT NULL,
  followup_id          BIGINT REFERENCES pharmacy_quote_followups(id) ON DELETE S
  trello_card_id       TEXT NOT NULL,
  trello_short_link    TEXT,
  trello_short_url     TEXT,
  trello_list_id       TEXT NOT NULL,
  trello_board_id      TEXT NOT NULL,
  sync_status          TEXT NOT NULL DEFAULT 'synced',
  -- synced | drift | error | deleted
  last_sync_at         TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  last_inbound_action  TEXT,
  last_inbound_at      TIMESTAMPTZ,
  last_payload         JSONB,
  external_id          UUID NOT NULL DEFAULT gen_random_uuid(),
  CONSTRAINT uq_trello_card UNIQUE (trello_card_id)
);
CREATE INDEX IF NOT EXISTS ix_trello_receita
  ON trello_cards_sync (clinica_id, receita_id);
CREATE INDEX IF NOT EXISTS ix_trello_status
  ON trello_cards_sync (clinica_id, sync_status);

-- Tabela de eventos inbound (dedup at-least-once)
CREATE TABLE IF NOT EXISTS trello_webhook_events (
  action_id   TEXT PRIMARY KEY,
  action_type TEXT NOT NULL,
  card_id     TEXT,
```

```
  received_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  processed_at TIMESTAMPTZ,
  payload     JSONB NOT NULL,
  status      TEXT NOT NULL DEFAULT 'pending'
);
CREATE INDEX IF NOT EXISTS ix_trello_evt_status
  ON trello_webhook_events (status, received_at);

-- 5) pharmacy_quality_score ------------------------------------------
CREATE TABLE IF NOT EXISTS pharmacy_quality_score (
  id                     BIGSERIAL PRIMARY KEY,
  clinica_id             INTEGER NOT NULL,
  farmacia_id            INTEGER NOT NULL,
  visita_dono            BOOLEAN NOT NULL DEFAULT FALSE,
  visita_data            DATE,
  nota_insumo            NUMERIC(2,1) CHECK (nota_insumo BETWEEN 0 AND 5),
  nota_atendimento       NUMERIC(2,1) CHECK (nota_atendimento BETWEEN 0 AND 5),
  nota_processamento     NUMERIC(2,1) CHECK (nota_processamento BETWEEN 0 AND 5),
  nota_comunicacao       NUMERIC(2,1) CHECK (nota_comunicacao BETWEEN 0 AND 5),
  nota_entrega           NUMERIC(2,1) CHECK (nota_entrega BETWEEN 0 AND 5),
  especialidades         TEXT[] NOT NULL DEFAULT '{}',
  -- ex: {hormonal, fitoterapico, suplemento, dermato, pediatria}
  ultima_reuniao_em      DATE,
  observacoes            TEXT,
  master_user_id         INTEGER,
  updated_at             TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  CONSTRAINT uq_quality_farmacia UNIQUE (clinica_id, farmacia_id)
);
CREATE INDEX IF NOT EXISTS ix_quality_visita
  ON pharmacy_quality_score (clinica_id, visita_dono);

-- View agregada (score composto ponderado)
CREATE OR REPLACE VIEW v_pharmacy_quality_score AS
SELECT
  id, clinica_id, farmacia_id, visita_dono, visita_data,
  nota_insumo, nota_atendimento, nota_processamento,
  nota_comunicacao, nota_entrega, especialidades, observacoes,
  ultima_reuniao_em, updated_at,
  ROUND(
    (COALESCE(nota_insumo,0)       * 0.30 +
     COALESCE(nota_atendimento,0)  * 0.20 +
     COALESCE(nota_processamento,0)* 0.20 +
     COALESCE(nota_comunicacao,0)  * 0.15 +
     COALESCE(nota_entrega,0)      * 0.15)::numeric, 2
  ) AS score_composto,
  CASE WHEN ultima_reuniao_em IS NULL THEN NULL
       ELSE (CURRENT_DATE - ultima_reuniao_em) END AS dias_sem_reuniao
```

```
FROM pharmacy_quality_score;

-- 6) payment_split_config -------------------------------------------
CREATE TABLE IF NOT EXISTS payment_split_config (
  id                          BIGSERIAL PRIMARY KEY,
  clinica_id                  INTEGER NOT NULL,
  farmacia_id                 INTEGER NOT NULL,
  gateway                     payment_gateway_kind NOT NULL DEFAULT 'asaas',
  wallet_id_externo           TEXT,
  recipient_id_externo        TEXT,
  percentual_clinica          NUMERIC(5,2) NOT NULL DEFAULT 0
                               CHECK (percentual_clinica BETWEEN 0 AND 100),
  percentual_servico_vendas   NUMERIC(5,2) NOT NULL DEFAULT 0
                               CHECK (percentual_servico_vendas BETWEEN 0 AND 100
  margem_desconto_autorizada  NUMERIC(5,2) NOT NULL DEFAULT 0
                               CHECK (margem_desconto_autorizada BETWEEN 0 AND 50
  desconto_maximo_sem_escalacao NUMERIC(5,2) NOT NULL DEFAULT 5,
  credenciais_enc             BYTEA,
  ativo                       BOOLEAN NOT NULL DEFAULT FALSE,
  created_at                  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  updated_at                  TIMESTAMPTZ NOT NULL DEFAULT NOW(),
  master_user_id              INTEGER,
  CONSTRAINT uq_split_farmacia UNIQUE (clinica_id, farmacia_id, gateway)
);
CREATE INDEX IF NOT EXISTS ix_split_ativo
  ON payment_split_config (clinica_id, ativo);

-- 7) agent_runtime_config (parametrização sem migration) --------------
-- Permite ao CEO master editar prazos, scripts, thresholds sem db:push.
CREATE TABLE IF NOT EXISTS agent_runtime_config (
  key            TEXT PRIMARY KEY,
  clinica_id     INTEGER NOT NULL,
  value          JSONB NOT NULL,
  description    TEXT,
  updated_by     INTEGER,
  updated_at     TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

INSERT INTO agent_runtime_config (key, clinica_id, value, description) VALUES
  ('rescue.prazo_redistribuicao_horas', 1, '48'::jsonb,
    'Horas sem contato antes de liberar redistribuição'),
  ('rescue.janela_contato_inicio',      1, '"09:00"'::jsonb,
    'Horário inicial permitido p/ contato WhatsApp'),
  ('rescue.janela_contato_fim',         1, '"20:00"'::jsonb,
    'Horário final permitido p/ contato WhatsApp'),
  ('agent.max_turnos_antes_escalar',    1, '5'::jsonb, NULL),
  ('agent.confidence_min_autonomo',     1, '0.75'::jsonb, NULL),
```

```
  ('agent.template_abertura',           1,
    '"Oi {nome}, aqui é {equipe} da clínica do Dr. Caio. Soube que a farmácia {fa
    'Tom tranquilo, sem urgência, curioso')
ON CONFLICT (key) DO NOTHING;

COMMIT;

-- Prova de aplicação: ------------------------------------------------
-- SELECT table_name FROM information_schema.tables
--  WHERE table_name IN (
--    'pharmacy_quote_followups','pharmacy_redistribution',
--    'agent_conversation_context','trello_cards_sync',
--    'trello_webhook_events','pharmacy_quality_score',
--    'payment_split_config','agent_runtime_config');
-- Esperado: 8 linhas.
```

Decisões de design destacadas. Toda tabela traz `clinica_id` na primeira coluna lógica, coerente com o contrato multi-tenant do MEDCORE. Todo campo monetário é `BIGINT` em centavos, zero float. `tentativas` em `JSONB` em vez de tabela filha porque o padrão é append-only raro, <20 linhas por receita, e permite consulta com operadores `@>` sem join. A view `v_pharmacy_quality_score` já calcula o score composto ponderado (insumo 30%, atendimento/processamento 20% cada, comunicação/entrega 15% cada) para consumo direto pelo radar. A tabela `agent_runtime_config` é a **chave da editabilidade sem migration**: CEO master abre painel, edita prazos, scripts e thresholds, e o worker lê `agent_runtime_config` no início de cada ciclo.

Seção 2 — Árvore de decisão do agente Claude Code (YAML estruturado)

```yaml
agent: pharmacy_conversion_agent
version: "1.0.0"
model: claude-sonnet-4
persona:
  nome: "equipe do Dr. Caio"
  tom: "tranquilo, curioso, sem urgência, sem ansiedade"
  regras_invioláveis:
    - "nunca dizer que a farmácia X 'não vendeu' ou 'falhou'"
    - "nunca pressionar o paciente"
    - "nunca mencionar comissão, margem de desconto, auditoria"
    - "sempre tratar por primeiro nome ou como o paciente se apresentou"
    - "janela de contato respeitada: 09:00-20:00 local do paciente"
    - "LGPD: nunca mencionar diagnóstico, dose, princípio ativo em canal externo"
```

```yaml
runtime_config_keys:
  - rescue.prazo_redistribuicao_horas
  - agent.max_turnos_antes_escalar
  - agent.confidence_min_autonomo
  - agent.template_abertura

initial_contact:
  trigger: "followup.status = 'enviada' AND NOW() - emitida_em >= prazo_redistrib
  canal: whatsapp
  message_template: "{{agent.template_abertura}}"
  # Resolve {nome}, {equipe}, {farmacia} via binding do worker
  expected_responses: [cotando, nao_recebeu_orcamento, motivo_financeiro, motivo_

intent_detection:
  # Classificação zero-shot do Claude com confidence explícita
  method: classify_with_confidence
  output_schema:
    intent: agent_intent
    confidence: float_0_1
    rationale: string
  fallback_se_confidence_baixa:
    threshold: "{{agent.confidence_min_autonomo}}"
    action: "pergunta_clarificacao_unica"
    max_retries: 1
    depois_disso: "escalar_humano"

responses:

  - intent: cotando
    action: modo_cotacao_sutil
    message: >
      Ah, entendi! Cotar é sempre bom mesmo. Se você quiser, eu posso ver com
      outras farmácias que a gente trabalha pra você ter mais opções de preço —
      todas de qualidade que o Dr. Caio conferiu pessoalmente. Quer que eu peça
      orçamento em mais uma ou duas?
    if_yes:
      action: redistribuir
      requires: pharmacy_redistribution.autorizado_por_master = true
      selecao_farmacias: auto_via_kaizen_recomendador
      notifica_master: true
    if_no:
      action: agenda_followup
      dias: 5
      mensagem: "Fechou! Qualquer coisa me chama. Te dou um toque daqui uns dias.

  - intent: nao_recebeu_orcamento
```

```
  action: ponte_reforcada_farmacia
  internal_steps:
    - cria_ticket_farmacia:
        canal: whatsapp_farmacia
        mensagem: "Oi, tudo bem? Sobre a receita #{receita_codigo} do paciente
    - sla_interno_horas: 24
    - timer: "se farmácia não responder em 24h → libera redistribuição automáti
  message_paciente: >
    Desculpa o aperto! Vou falar com a farmácia agora pra acelerar o seu
    orçamento. Volto a te avisar ainda hoje.
  auto_action: cria_card_trello_com_label("ponte_ativa")

- intent: motivo_financeiro
  action: modo_negociacao_margem
  requires_config: payment_split_config.ativo = true
  guardrails:
    - "desconto solicitado <= payment_split_config.margem_desconto_autorizada"
    - "se desconto > desconto_maximo_sem_escalacao → escalar_humano"
  message_inicial: >
    Entendi, e é super válido! Eu consigo conversar com a farmácia pra buscar
    uma condição melhor pra você. Me conta: qual faixa de valor funcionaria
    melhor agora?
  fluxo:
    - captura_valor_alvo_paciente
    - calcula_desconto_necessario
    - se_dentro_da_margem:
        aciona: gateway_split_com_desconto
        mensagem: "Consegui! Com a farmácia, sai por R$ {valor_ajustado}. Posso
    - senao:
        aciona: escalar_humano
        motivo: "desconto solicitado fora da margem pré-autorizada"

- intent: motivo_tempo
  action: agenda_followup
  message: >
    Sem problema nenhum! Vou te reenviar em uma semana, tá? Se quiser
    adiantar antes, me avisa aqui.
  auto_action:
    - cria_card_trello_label("followup_agendado")
    - agenda_task: "pharmacy_rescue.followup_retry"
    - dias: 7

- intent: nao_quer_mais
  action: encerrar_respeitoso
  message: "Sem problema, viu! Se mudar de ideia ou precisar de qualquer coisa,
  persist: { status: cancelada, motivo: "paciente declinou" }
```

```
  - intent: ja_fechou_outra
    action: registrar_perda_amigavel
    message: "Ótimo que resolveu! Só me conta, se puder: foi em farmácia nossa pa
    telemetria: registrar_perda(motivo="fechou_fora")

  - intent: escalar_humano
    action: hand_off_humano
    card_trello:
      label: "urgente_humano"
      checklist:
        - "Ler toda a thread (link interno)"
        - "Ligar para o paciente"
        - "Decidir: redistribuir | desconto extra | encerrar"
      due_em_horas: 4

escalation_matrix:
  escalar_sempre_se:
    - "paciente menciona efeito adverso ou reação"
    - "paciente menciona morte, luto, hospitalização"
    - "paciente reclama formalmente da clínica ou do médico"
    - "confidence < threshold por 2 turnos consecutivos"
    - "turnos > agent.max_turnos_antes_escalar"
    - "intenção ambígua + valor estimado > R$ 1.500"

telemetria_obrigatoria:
  por_turno:
    - agent_conversation_context insere uma linha
    - tokens_input, tokens_output, model_version
  por_fluxo:
    - atualiza pharmacy_quote_followups.status
    - atualiza pharmacy_quote_followups.tentativas (append)
```

Por que tranquilo funciona melhor do que agressivo. Pacientes de clínica integrativa vêm com perfil de consumidor cuidadoso, que escolhe o médico por vínculo de confiança. Pressão quebra vínculo. O agente finge ser “ajudante natural da equipe” — e é exatamente isso que ele é, do ponto de vista técnico: uma extensão do secretariado, com memória e paciência infinita. A escala humana só acontece quando há sinal clínico, quando o paciente escorrega pra emocional, ou quando o valor justifica o custo humano.

Seção 3 — Endpoints TypeScript prontos pra colar

Arquivo base: `server/routes/pharmacy-rescue.ts`. Todos os endpoints abaixo usam o middleware duplo `validador_mestre` + `requireMasterEstrito` (já existente no MEDCORE),

retornam shape claro, têm `try/catch` defensivo que nunca derruba o fluxo principal, e são idempotentes por design (chave natural + `ON CONFLICT` / `UPDATE` sem side-effect duplo).

```
// server/routes/pharmacy-rescue.ts
import { Router, Request, Response } from "express";
import { z } from "zod";
import { db } from "../db";
import { validador_mestre, requireMasterEstrito } from "../middleware/auth";
import { TrelloClient } from "../integrations/trello/client";
import { enqueueAgentTurn } from "../workers/agent-queue";
import { loadRuntimeConfig } from "../services/runtime-config";
import { recomendarFarmacias } from "../services/pharmacy-recommender";
import { createChargeWithSplit } from "../integrations/asaas/payments";

const router = Router();
router.use(validador_mestre);
router.use(requireMasterEstrito);

// Padrão de resposta ----------------------------------------------------
type Ok<T>  = { ok: true; data: T };
type Err    = { ok: false; error: string; hint?: string };
const ok  = <T,>(data: T): Ok<T> => ({ ok: true, data });
const err = (msg: string, hint?: string): Err => ({ ok: false, error: msg, hint }

// 1) Worker de detecção de followups vencidos -------------------------
router.post("/trigger-followup-check", async (req, res) => {
  try {
    const clinica_id = req.user!.clinica_id;
    const cfg = await loadRuntimeConfig(clinica_id);
    const prazoH = cfg["rescue.prazo_redistribuicao_horas"] ?? 48;

    const result = await db.$queryRaw<{ id: number; receita_id: number }[]>`
      UPDATE pharmacy_quote_followups
         SET status = 'sem_contato',
             liberado_redistribuir_em = NOW(),
             ultima_atualizacao_em    = NOW()
       WHERE clinica_id = ${clinica_id}
         AND status IN ('enviada','aguardando_contato')
         AND NOW() - emitida_em >= make_interval(hours => ${prazoH})
         AND (liberado_redistribuir_em IS NULL)
      RETURNING id, receita_id;
    `;

    // Enfileira agente para cada followup liberado
    for (const r of result) {
      await enqueueAgentTurn({ followup_id: r.id, turno: 0, clinica_id });
```

```
    }

    return res.json(ok({ liberados: result.length, followups: result }));
  } catch (e: any) {
    console.error("[rescue.trigger-followup]", e);
    return res.status(500).json(err("falha ao processar followup check", e.messag
  }
});

// 2) Candidatos à redistribuição -------------------------------------
router.get("/candidates-for-redistribution", async (req, res) => {
  try {
    const clinica_id = req.user!.clinica_id;
    const rows = await db.$queryRaw<any[]>`
      SELECT f.id, f.receita_id, f.paciente_id, f.farmacia_original_id,
             f.valor_estimado_centavos, f.status, f.emitida_em,
             f.liberado_redistribuir_em,
             p.nome_iniciais,
             far.nome AS farmacia_original_nome,
             EXTRACT(EPOCH FROM (NOW() - f.emitida_em))/3600 AS horas_desde_emiss
        FROM pharmacy_quote_followups f
        JOIN pacientes_view p   ON p.id = f.paciente_id
        JOIN farmacias    far   ON far.id = f.farmacia_original_id
       WHERE f.clinica_id = ${clinica_id}
         AND f.status IN ('sem_contato','aguardando_contato')
         AND f.liberado_redistribuir_em IS NOT NULL
       ORDER BY f.valor_estimado_centavos DESC, f.emitida_em ASC
       LIMIT 200;`;
    return res.json(ok({ total: rows.length, candidatos: rows }));
  } catch (e: any) {
    console.error("[rescue.candidates]", e);
    return res.status(500).json(err("falha ao listar candidatos"));
  }
});

// 3) Redistribuir receita ---------------------------------------------
const redistribuirSchema = z.object({
  followup_id: z.number().int().positive(),
  farmacias_destino: z.array(z.number().int().positive()).min(1).max(5),
  motivo: z.string().min(3).max(500),
  mensagem_paciente: z.string().max(1000).optional(),
});
router.post("/redistribute", async (req, res) => {
  try {
    const input = redistribuirSchema.parse(req.body);
    const clinica_id = req.user!.clinica_id;
    const master_user_id = req.user!.id;
```

```
    const row = await db.$queryRaw<any[]>`
      SELECT id, receita_id, status, farmacia_original_id
        FROM pharmacy_quote_followups
       WHERE id = ${input.followup_id} AND clinica_id = ${clinica_id}
       FOR UPDATE;`;
    if (!row[0]) return res.status(404).json(err("followup não encontrado"));
    if (row[0].status === "fechada")
      return res.status(409).json(err("receita já fechada, não redistribui"));

    if (input.farmacias_destino.includes(row[0].farmacia_original_id))
      return res.status(400).json(err("não redistribua para a farmácia original")

    const statusCada = Object.fromEntries(
      input.farmacias_destino.map(fid => [fid, {
        status: "enviada", enviada_em: new Date().toISOString(), resposta: null,
      }])
    );

    const [redist] = await db.$queryRaw<any[]>`
      INSERT INTO pharmacy_redistribution (
        clinica_id, followup_id, receita_id, farmacias_destino, motivo,
        mensagem_paciente, autorizado_por_master, master_user_id, status_cada_far
      ) VALUES (
        ${clinica_id}, ${input.followup_id}, ${row[0].receita_id},
        ${input.farmacias_destino}::int[], ${input.motivo},
        ${input.mensagem_paciente ?? null}, TRUE, ${master_user_id},
        ${JSON.stringify(statusCada)}::jsonb
      ) RETURNING id, criada_em;`;

    await db.$executeRaw`
      UPDATE pharmacy_quote_followups
         SET status = 'redistribuida', ultima_atualizacao_em = NOW()
       WHERE id = ${input.followup_id};`;

    return res.json(ok({ redistribuicao_id: redist.id, criada_em: redist.criada_e
  } catch (e: any) {
    if (e instanceof z.ZodError) return res.status(400).json(err("payload inválid
    console.error("[rescue.redistribute]", e);
    return res.status(500).json(err("falha ao redistribuir"));
  }
});

// 4) Início da conversa do agente -------------------------------------
const startSchema = z.object({ followup_id: z.number().int().positive() });
router.post("/agent-conversation/start", async (req, res) => {
  try {
```

```
    const { followup_id } = startSchema.parse(req.body);
    const clinica_id = req.user!.clinica_id;
    const cfg = await loadRuntimeConfig(clinica_id);

    const [fu] = await db.$queryRaw<any[]>`
      SELECT f.id, f.paciente_id, f.receita_id,
             p.primeiro_nome, far.nome_curto AS farmacia
        FROM pharmacy_quote_followups f
        JOIN pacientes_view p  ON p.id = f.paciente_id
        JOIN farmacias far     ON far.id = f.farmacia_original_id
       WHERE f.id = ${followup_id} AND f.clinica_id = ${clinica_id};`;
    if (!fu) return res.status(404).json(err("followup não encontrado"));

    const thread_id = `fu-${followup_id}-${Date.now()}`;
    const template: string = cfg["agent.template_abertura"];
    const mensagem = template
      .replace("{nome}", fu.primeiro_nome)
      .replace("{equipe}", "equipe do Dr. Caio")
      .replace("{farmacia}", fu.farmacia);

    await db.$executeRaw`
      INSERT INTO agent_conversation_context
        (clinica_id, followup_id, paciente_id, thread_id, turno, mensagem_agente)
      VALUES (${clinica_id}, ${followup_id}, ${fu.paciente_id},
              ${thread_id}, 0, ${mensagem});`;

    await enqueueAgentTurn({ thread_id, canal: "whatsapp", mensagem, paciente_id:

    return res.json(ok({ thread_id, mensagem, turno: 0 }));
  } catch (e: any) {
    console.error("[agent.start]", e);
    return res.status(500).json(err("falha ao iniciar conversa do agente"));
  }
});

// 5) Recebimento de resposta + detecção de intent ---------------------
const intentSchema = z.object({
  thread_id: z.string().min(5),
  resposta_paciente: z.string().min(1).max(4000),
});
router.post("/agent-conversation/intent", async (req, res) => {
  try {
    const { thread_id, resposta_paciente } = intentSchema.parse(req.body);
    const clinica_id = req.user!.clinica_id;

    // Carrega contexto da thread
    const turnos = await db.$queryRaw<any[]>`
```

```
      SELECT turno, mensagem_agente, resposta_paciente, intent_detectado
        FROM agent_conversation_context
       WHERE thread_id = ${thread_id} AND clinica_id = ${clinica_id}
       ORDER BY turno ASC;`;
    if (!turnos.length) return res.status(404).json(err("thread não encontrada"))
    const ultimoTurno = turnos[turnos.length - 1].turno as number;

    // Classificação (chamada ao Claude, função encapsulada)
    const { intent, confidence, rationale, proxima_mensagem, decisao } =
      await classifyAndRespond({ thread_id, historico: turnos, resposta_paciente,

    const cfg = await loadRuntimeConfig(clinica_id);
    const minConf = cfg["agent.confidence_min_autonomo"] ?? 0.75;
    const escalado = confidence < minConf || intent === "escalar_humano";

    await db.$executeRaw`
      INSERT INTO agent_conversation_context (
        clinica_id, followup_id, paciente_id, thread_id, turno,
        mensagem_agente, resposta_paciente, intent_detectado, confidence,
        decisao_proximo, escalado_humano, motivo_escalacao
      ) SELECT ${clinica_id}, followup_id, paciente_id, ${thread_id},
               ${ultimoTurno + 1}, ${proxima_mensagem}, ${resposta_paciente},
               ${intent}::agent_intent, ${confidence}, ${decisao},
               ${escalado}, ${escalado ? rationale : null}
          FROM agent_conversation_context
         WHERE thread_id = ${thread_id} LIMIT 1;`;

    return res.json(ok({
      intent, confidence, rationale,
      proxima_mensagem, decisao, escalado,
      turno: ultimoTurno + 1,
    }));
  } catch (e: any) {
    console.error("[agent.intent]", e);
    return res.status(500).json(err("falha ao processar resposta"));
  }
});

// 6) Ponte reforçada com a farmácia -----------------------------------
const bridgeSchema = z.object({
  followup_id: z.number().int().positive(),
  mensagem_farmacia: z.string().min(10).max(2000),
});
router.post("/pharmacy-bridge", async (req, res) => {
  try {
    const { followup_id, mensagem_farmacia } = bridgeSchema.parse(req.body);
    const clinica_id = req.user!.clinica_id;
```

```
    const tentativa = {
      data: new Date().toISOString(),
      canal: "whatsapp_farmacia",
      resultado: "enviada",
      mensagem: mensagem_farmacia,
      operador: "agente_claude_code",
    };

    await db.$executeRaw`
      UPDATE pharmacy_quote_followups
         SET tentativas = tentativas || ${JSON.stringify([tentativa])}::jsonb,
             status = 'aguardando_contato',
             ultima_atualizacao_em = NOW()
       WHERE id = ${followup_id} AND clinica_id = ${clinica_id};`;

    // Dispara mensagem real via provider WhatsApp Business (encapsulado)
    // await whatsappClient.sendToFarmacia(...);

    return res.json(ok({ followup_id, sla_horas: 24 }));
  } catch (e: any) {
    console.error("[rescue.bridge]", e);
    return res.status(500).json(err("falha na ponte com farmácia"));
  }
});

// 7) Trello — criar card com 5 campos completos ------------------------
const cardSchema = z.object({
  followup_id: z.number().int().positive(),
  urgencia: z.enum(["normal","alta","critica"]).default("normal"),
});
router.post("/../trello-sync/create-card", async (req, res) => {
  try {
    const { followup_id, urgencia } = cardSchema.parse(req.body);
    const clinica_id = req.user!.clinica_id;

    const [fu] = await db.$queryRaw<any[]>`
      SELECT f.id, f.receita_id, f.paciente_id, f.valor_estimado_centavos,
             p.iniciais, r.codigo_formula, r.patologia_codigo,
             far.nome_curto AS farmacia_nome, far.id AS farmacia_id,
             qs.score_composto, qs.visita_dono,
             agg.ultima_resposta, agg.tentativas_count
        FROM pharmacy_quote_followups f
        JOIN pacientes_view p         ON p.id = f.paciente_id
        JOIN receitas r               ON r.id = f.receita_id
        JOIN farmacias far            ON far.id = f.farmacia_original_id
        LEFT JOIN v_pharmacy_quality_score qs
```

```
          ON qs.farmacia_id = f.farmacia_original_id
         AND qs.clinica_id  = f.clinica_id
    LEFT JOIN LATERAL (
      SELECT COUNT(*) AS tentativas_count,
             MAX((t->>'data')::timestamptz) AS ultima_resposta
        FROM jsonb_array_elements(f.tentativas) t
    ) agg ON TRUE
   WHERE f.id = ${followup_id} AND f.clinica_id = ${clinica_id};`;
if (!fu) return res.status(404).json(err("followup não encontrado"));

const trello = new TrelloClient({
  apiKey: process.env.TRELLO_API_KEY!,
  token : process.env.TRELLO_TOKEN!,
  clientIdentifier: process.env.TRELLO_SELF_IDENTIFIER,
});

const labelMap: Record<string,string> = {
  normal: process.env.TRELLO_LABEL_NORMAL!,
  alta:   process.env.TRELLO_LABEL_ALTA!,
  critica:process.env.TRELLO_LABEL_CRITICA!,
};

const desc = [
  `**Receita:** \`${fu.codigo_formula}\`  ·  **Patologia:** \`${fu.patologia_
  `**Valor estimado:** R$ ${(fu.valor_estimado_centavos/100).toFixed(2)}`,
  `**Farmácia original:** ${fu.farmacia_nome}  ·  **Score:** ${fu.score_compo
  ``,
  `**Tentativas de contato:** ${fu.tentativas_count ?? 0}  ·  **Última respos
  ``,
  `🔗 [Abrir no PAWARDS](${process.env.APP_URL}/admin/pharmacy-rescue/${follc
  ``,
  `> LGPD: dados clínicos completos permanecem apenas no sistema interno.`,
].join("\n");

const card = await trello.createCard({
  idList: process.env.TRELLO_LIST_INBOX!,
  name  : `${fu.iniciais} · ${fu.codigo_formula} · R$${(fu.valor_estimado_cen
  desc,
  idLabels: [labelMap[urgencia]],
  due: new Date(Date.now() + 48*3600*1000).toISOString(),
});

await trello.addChecklist(card.id, "Próximas ações").then(async (cl) => {
  await trello.addCheckItem(cl.id, "Contato WhatsApp com paciente (1ª tentati
  await trello.addCheckItem(cl.id, "Ler argumentos de conversão sugeridos pel
  await trello.addCheckItem(cl.id, "Decidir: redistribuir ou escalar");
});
```

```
    await db.$executeRaw`
      INSERT INTO trello_cards_sync (
        clinica_id, receita_id, followup_id,
        trello_card_id, trello_short_link, trello_short_url,
        trello_list_id, trello_board_id, last_payload
      ) VALUES (
        ${clinica_id}, ${fu.receita_id}, ${followup_id},
        ${card.id}, ${card.shortLink}, ${card.shortUrl},
        ${card.idList}, ${card.idBoard}, ${JSON.stringify(card)}::jsonb
      ) ON CONFLICT (trello_card_id) DO UPDATE
        SET last_sync_at = NOW(), last_payload = EXCLUDED.last_payload;`;

    return res.json(ok({
      trello_card_id: card.id, shortUrl: card.shortUrl,
    }));
  } catch (e: any) {
    console.error("[trello.create-card]", e);
    return res.status(500).json(err("falha ao criar card Trello"));
  }
});

// 8) Edição de quality score ----------------------------------------
const scorePatch = z.object({
  visita_dono: z.boolean().optional(),
  visita_data: z.string().optional(),
  nota_insumo: z.number().min(0).max(5).optional(),
  nota_atendimento: z.number().min(0).max(5).optional(),
  nota_processamento: z.number().min(0).max(5).optional(),
  nota_comunicacao: z.number().min(0).max(5).optional(),
  nota_entrega: z.number().min(0).max(5).optional(),
  especialidades: z.array(z.string()).optional(),
  observacoes: z.string().max(2000).optional(),
});
router.patch("/../pharmacy-quality-score/:farmacia_id", async (req, res) => {
  try {
    const farmacia_id = Number(req.params.farmacia_id);
    const patch = scorePatch.parse(req.body);
    const clinica_id = req.user!.clinica_id;
    const master_user_id = req.user!.id;

    const [row] = await db.$queryRaw<any[]>`
      INSERT INTO pharmacy_quality_score (
        clinica_id, farmacia_id, visita_dono, visita_data,
        nota_insumo, nota_atendimento, nota_processamento,
        nota_comunicacao, nota_entrega, especialidades,
        observacoes, master_user_id, updated_at
```

```
      ) VALUES (
        ${clinica_id}, ${farmacia_id},
        ${patch.visita_dono ?? false}, ${patch.visita_data ?? null},
        ${patch.nota_insumo ?? null}, ${patch.nota_atendimento ?? null},
        ${patch.nota_processamento ?? null}, ${patch.nota_comunicacao ?? null},
        ${patch.nota_entrega ?? null}, ${patch.especialidades ?? []}::text[],
        ${patch.observacoes ?? null}, ${master_user_id}, NOW()
      )
      ON CONFLICT (clinica_id, farmacia_id) DO UPDATE SET
        visita_dono        = COALESCE(EXCLUDED.visita_dono, pharmacy_quality_scor
        visita_data        = COALESCE(EXCLUDED.visita_data, pharmacy_quality_scor
        nota_insumo        = COALESCE(EXCLUDED.nota_insumo, pharmacy_quality_scor
        nota_atendimento   = COALESCE(EXCLUDED.nota_atendimento, pharmacy_quality
        nota_processamento = COALESCE(EXCLUDED.nota_processamento, pharmacy_quali
        nota_comunicacao   = COALESCE(EXCLUDED.nota_comunicacao, pharmacy_quality
        nota_entrega       = COALESCE(EXCLUDED.nota_entrega, pharmacy_quality_sco
        especialidades     = COALESCE(EXCLUDED.especialidades, pharmacy_quality_s
        observacoes        = COALESCE(EXCLUDED.observacoes, pharmacy_quality_scor
        master_user_id     = EXCLUDED.master_user_id,
        updated_at         = NOW()
      RETURNING *;`;

    return res.json(ok(row));
  } catch (e: any) {
    if (e instanceof z.ZodError) return res.status(400).json(err("payload inválid
    console.error("[quality.patch]", e);
    return res.status(500).json(err("falha ao editar score"));
  }
});

export default router;
```

Auxiliar `classifyAndRespond` (arquivo separado, Claude Sonnet 4):

```
// server/services/agent-classifier.ts
import Anthropic from "@anthropic-ai/sdk";
import { loadRuntimeConfig } from "./runtime-config";

const client = new Anthropic({ apiKey: process.env.ANTHROPIC_API_KEY! });

export async function classifyAndRespond(args: {
  thread_id: string;
  historico: Array<{ turno:number; mensagem_agente:string|null; resposta_paciente
  resposta_paciente: string;
  clinica_id: number;
}) {
```

```
  const cfg = await loadRuntimeConfig(args.clinica_id);

  const systemPrompt = `
Você é a equipe tranquila do Dr. Caio, clínica integrativa.
Tom: tranquilo, curioso, sem urgência, sem ansiedade.
Nunca mencione: comissão, margem, auditoria, "não vendeu", diagnóstico, dose, pri
Respeite LGPD: nunca nomeie patologia ou medicação em canal externo.
Classifique a intenção do paciente em EXATAMENTE UM de:
  cotando | nao_recebeu_orcamento | motivo_financeiro | motivo_tempo | nao_quer_m
Devolva JSON estrito: { intent, confidence, rationale, proxima_mensagem, decisao
`.trim();

  const userPrompt = JSON.stringify({ historico: args.historico, resposta: args.r

  const resp = await client.messages.create({
    model: "claude-sonnet-4-5",
    max_tokens: 800,
    system: systemPrompt,
    messages: [{ role: "user", content: userPrompt }],
  });

  const text = resp.content.filter(c => c.type === "text").map((c: any) => c.text
  const json = text.slice(text.indexOf("{"), text.lastIndexOf("}")+1);
  const parsed = JSON.parse(json);

  return {
    intent: parsed.intent as
      "cotando"|"nao_recebeu_orcamento"|"motivo_financeiro"|"motivo_tempo"|
      "nao_quer_mais"|"ja_fechou_outra"|"escalar_humano"|"indefinido",
    confidence: Number(parsed.confidence ?? 0),
    rationale: String(parsed.rationale ?? ""),
    proxima_mensagem: String(parsed.proxima_mensagem ?? ""),
    decisao: String(parsed.decisao ?? ""),
  };
}
```

Validação inbound Trello (reuso e ponte bidirecional), `server/routes/trello-webhook.ts`:

```
import express, { Request, Response } from "express";
import crypto from "node:crypto";
export const trelloWebhook = express.Router();

trelloWebhook.head("/trello/webhook", (_req, res) => res.status(200).end());

trelloWebhook.post("/trello/webhook",
```

```
  express.json({ verify: (req: any,_r,buf)=>{ req.rawBody = Buffer.from(buf); } }
  async (req: any, res: Response) => {
    const secret = process.env.TRELLO_CLIENT_SECRET!;
    const cbURL  = process.env.TRELLO_CALLBACK_URL!;
    const sig    = req.get("X-Trello-Webhook");
    if (!sig || !req.rawBody) return res.status(401).end();
    const expected = crypto.createHmac("sha1", secret)
      .update(Buffer.concat([req.rawBody, Buffer.from(cbURL,"utf8")]))
      .digest("base64");
    const a = Buffer.from(sig,"utf8"), b = Buffer.from(expected,"utf8");
    if (a.length !== b.length || !crypto.timingSafeEqual(a,b))
      return res.status(401).end();

    const actionId = req.body?.action?.id;
    if (!actionId) return res.status(200).end();

    try {
      await db.$executeRaw`
        INSERT INTO trello_webhook_events(action_id, action_type, card_id, payloa
        VALUES (${actionId}, ${req.body.action.type},
                ${req.body.action.data?.card?.id ?? null},
                ${JSON.stringify(req.body)}::jsonb)
        ON CONFLICT (action_id) DO NOTHING;`;
    } catch(e) { console.error("[trello-webhook]", e); }
    return res.status(200).end();
  }
);
```

Seção 4 — Wireframes TDAH/TOC-friendly

Paleta: navy `#020406`, gold `#C89B3C`, semáforo `#16a34a` / `#d97706` / `#dc2626`. Fonte `Inter` em todos os pesos. Grid 12 colunas, gap 16, padding card 24. Gridlines pontilhados (`stroke-dasharray: 4 4`, cor `#1f2937` no dark e `#e5e7eb` no light). **Sempre**: porcentagem + número absoluto juntos, mesmo em labels de eixo.

4.1 Dashboard "Dinheiro na Mesa"

```
╔═══════════════════════════════════════════════════════════════════════════╗
║  PAWARDS MEDCORE · Pharmacy Rescue · Dinheiro na Mesa          [🔔] [CEO] ║
╟═══════════════════════════════════════════════════════════════════════════╢
║                                                                           ║
║  ┌─────────────────┐ ┌─────────────────┐ ┌─────────────────┐ ┌──────────────┐ ║
║  │ RECEITAS ABERTAS│ │ VALOR EM RISCO  │ │ CONVERSÃO ATUAL│ │ POTENCIAL    │║
```

```
│  │                  │ │                  │ │                  │ │ C/ RESCUE    │
│  │  31,8%           │ │ R$ 142.307,00    │ │  68,2%           │ │  81,4%       │
│  │ (1.384/4.357)    │ │ — estimado       │ │ (2.973/4.357)    │ │ (+570 rec.)  │
│  └──────────────────┘ └──────────────────┘ └──────────────────┘ └──────────────┘ │
│                                                                                   │
│  CONVERSÃO POR FARMÁCIA (top 10)                                                  │
│  ┌──────────────────────────────────────────────────────────────────────────────┐ │
│  │ Farma Alpha   ═══════════════════════ 82,1% (452/550)      🟢                │ │
│  │ Manipula Co.  ═══════════════════ 71,4% (320/448)          🟢                │ │
│  │ VitaFórmula   ═══════════════ 54,3% (187/344)              🟠                │ │
│  │ PhytoLab      ═══════════ 41,2% (98/238)                   🟠                │ │
│  │ GreenBio      ═══════ 19,7% (32/163)    🔴  ⚠ queda 28% ↓ 30d              │ │
│  │ ... ┆ gridlines verticais pontilhados a cada 20% ┆                           │ │
│  └──────────────────────────────────────────────────────────────────────────────┘ │
│                                                                                   │
│  DINHEIRO POR FAIXA ETÁRIA DA RECEITA  ·  COM/SEM RESCUE (projeção 30d)           │
│  ┌──────────────────────────┐  ┌──────────────────────────────────────────────┐ │
│  │ 0-48h     ══ R$ 18 k     │  │ Cenário atual:  R$ 142.307,00 em aberto      │ │
│  │ 48h-7d    ═══ R$ 47 k    │  │ Com rescue 30d: +R$ 64.210,00 (+45,1%)       │ │
│  │ 7-30d     ═══ R$ 62 k    │  │ ──────────────────────────                   │ │
│  │ >30d      ══ R$ 15 k     │  │ Receitas elegíveis HOJE: 204                 │ │
│  └──────────────────────────┘  └──────────────────────────────────────────────┘ │
│                                                                                   │
│  PAWARDS® Clinical Intelligence System · Engineered by PADCON® Advanced           │
│  Systems Architecture                                                             │
└───────────────────────────────────────────────────────────────────────────────────┘
```

Regra de cores nas barras horizontais: `conversao_pct < 30% → #dc2626` vermelho, `30-60% → #d97706` âmbar, `>60% → #16a34a` verde. Gradiente sutil dentro da barra (180° left→right, 20% de brilho a mais no fim) para dar sensação de "preenchimento". Cada barra carrega tooltip com 5 infos: receitas totais, fechadas, em aberto, valor médio, tempo médio até fechar.

4.2 Painel de redistribuição

```
┌───────────────────────────────────────────────────────────────────────────────┐
│  Redistribuir receita #R-8743  ·  J.S. · R$ 287,00 · 52h sem contato          │
├───────────────────────────────────────────────────────────────────────────────┤
│  Farmácia original: VitaFórmula   Score composto: 3,2/5  🟠                  │
│                                                                               │
│  Selecione farmácias parceiras p/ redistribuir (min 1, max 5):                │
│  ┌──────────────────────────────────────────────────────────────────────────┐ │
│  │ ☑ Farma Alpha      ⭐4,8  ✅visita dono  🧪hormonal  📈conv 82%          │ │
│  │ ☑ Manipula Co.     ⭐4,5  ✅visita dono  💊suplemento  📈conv 71%        │ │
│  │ ☐ PhytoLab         ⭐3,9  ⚠sem visita    🌿fito       📈conv 41%         │ │
```

```
│ □ GreenBio          ⭐2,4  ⚠sem visita    ❗queda 28%   📉conv 19%   │
└──────────────────────────────────────────────────────────────────────┘

💡 Recomendação IA: Alpha + Manipula (kaizen #2 — match especialidade)

Preview da mensagem ao paciente (tom tranquilo):
┌──────────────────────────────────────────────────────────────────────┐
│ "Oi J., cotar é sempre bom mesmo. Separei 2 farmácias nossas pra     │
│  te mandar um orçamento ágil — ambas o Dr. Caio conhece              │
│  pessoalmente. Quando chegarem, te aviso por aqui 🙏"                 │
└──────────────────────────────────────────────────────────────────────┘
                        [ Cancelar ]  [ Redistribuir para 2 farmácias ]
```

4.3 Card do agente Claude Code

```
Agente Claude Code · Thread fu-8743-1714500000
Paciente: J.S.  ·  Turno: 3/5  ·  Intent atual: 🟡 cotando (0.82)

Conversa (tempo real):
┌──────────────────────────────────────────────────────────────────────┐
│ 🟦 Agente [14:02]  Oi J., tudo bem? Aqui é a equipe do Dr. Caio...    │
│ 🟨 J.S.   [14:07]  Oi! Tô cotando ainda, vou ver com mais uma farmác│
│ 🟦 Agente [14:07]  Ah, entendi! Cotar é sempre bom. Posso ver com    │
│                    outras farmácias nossas pra te dar mais opções?    │
│ 🟨 J.S.   [14:11]  Pode sim, obrigada!                                │
│ 🟦 Agente [14:11]  Perfeito! Daqui uns minutos te aviso 🙏            │
└──────────────────────────────────────────────────────────────────────┘

Intent detectado: cotando   Confiança: 82% (acima do threshold 75%)
[🟢 conversão]  [🟡 investigando]  [🟠 atenção]  [🔴 escalar humano]

Histórico de contato (tentativas): 3
  · 2026-04-22 10:00 · WhatsApp · sem_resposta
  · 2026-04-23 14:00 · WhatsApp · sem_resposta
  · 2026-04-24 14:02 · WhatsApp · respondeu ✅

Próxima ação sugerida pelo agente:
  → Disparar redistribuição para Farma Alpha + Manipula Co.
  → Criar card Trello com label "cotando_ativo"

        [ ❌ Rejeitar ]  [ ✏ Editar ]  [ ✅ Aprovar e executar ]
```

4.4 Score de qualidade das farmácias (radar)

```
╔═══════════════════════════════════════════════════════════════════════╗
║  Qualidade das Farmácias Parceiras  ·  Radar Multi-dimensional        ╒
╠═══════════════════════════════════════════════════════════════════════╣
║  Selecione farmácia: [Farma Alpha ▾]    Visita dono: ✅  (2026-01-15)  ╒
║                                                                       ╒
║                          Insumo (4,8)                                 ╒
║                         /     ┆     \                                 ╒
║                        /  ┆   ┆   ┆  \                              ╒
║          Entrega    ▲ ·····●——●——● ·····▲  Atendimento                ╒
║           (4,4)    /  ┆   ┆   ┆   ┆   ┆  \     (4,7)                 ╒
║                   /   ┆ • ┆   ┆   ┆ • ┆   \                          ╒
║                  /    ┆   ┆   ┆   ┆   ┆    \                         ╒
║                 ●————————————————————————●                            ╒
║        Comunicação (4,3)                Processamento (4,6)            ╒
║                                                                       ╒
║  Score composto: 4,62 / 5  🟢                                         ╒
║  Especialidades: hormonal, dermato, fitoterápico                      ╒
║  Última reunião presencial: 2026-03-02  (53 dias atrás) ✅            ╒
║                                                                       ╒
║  Editar (inline):                                                     ╒
║    Insumo         [ 4.8 ▾ ]   Atendimento     [ 4.7 ▾ ]              ╒
║    Processamento  [ 4.6 ▾ ]   Comunicação     [ 4.3 ▾ ]              ╒
║    Entrega        [ 4.4 ▾ ]   Observação:  [_________________]        ╒
║                                             [ Salvar ]                ╒
╚═══════════════════════════════════════════════════════════════════════╝
```

Regras universais de frontend TDAH/TOC-friendly. Tudo em grid 12-col com `align-items: baseline` — nada solto. Títulos de cards em 14px semibold navy, números-chave em 28px bold gold `#C89B3C`, labels secundárias em 12px slate `#64748B`. Toda porcentagem carrega `(absoluto/total)` na mesma linha, sem exceção. Gráficos usam `<svg>` com `<line stroke-dasharray='4 4'>` para gridlines verticais a cada 20%. Cor é redundante: sempre que usar vermelho, adicionar ícone ⚠ ou 🔴, nunca cor sozinha (acessibilidade + TOC). Hover state sempre troca opacidade (+10%) sem mover elemento — previne ansiedade de layout shift.

Seção 5 — Kaizens pré-autorizados (Dr. Replit implementa sem perguntar)

1. **Score preditivo de conversão por paciente.** Função `calcScoreConversao(paciente_id)` cruzando: (a) histórico de fechamentos anteriores na clínica, (b) tempo médio desde emissão até fechar, (c) valor médio das fórmulas

fechadas, (d) recência da última consulta, (e) canal de contato preferido do paciente. Saída: 0–100. Persiste em view materializada `mv_paciente_score_conversao` atualizada a cada 6h.

2. **Recomendador de farmácia para redistribuição.** Serviço `recomendarFarmacias(receita_id, top_n=2)` que combina `pharmacy_quality_score.score_composto` × matching de especialidade (fórmula hormonal → farmácia com tag `hormonal`) × taxa de conversão histórica da farmácia com pacientes de perfil similar (cluster por faixa etária e valor típico). Saída ordenada pela probabilidade esperada de fechamento.

3. **Detector de "farmácia fantasma".** Worker diário que calcula, por farmácia, a taxa de "receitas emitidas → paciente declarou `nao_recebeu_orcamento`" nas últimas 30d. Se >15%, gera alerta no painel CEO com label vermelho `farmacia_fantasma_suspeita`. Cruza com `pharmacy_quality_score.nota_comunicacao` para confirmar padrão.

4. **Heatmap de horários ótimos de contato.** Agregação sobre `agent_conversation_context.created_at` filtrando linhas com `resposta_paciente IS NOT NULL`. Grid 7 dias × 24h. Célula com mais respostas = verde intenso. Worker expõe `GET /api/admin/analytics/heatmap-contato`. O agente passa a usar esse heatmap para priorizar horário da 1ª mensagem.

5. **Dashboard preditivo de recuperação.** Card no topo do painel com mensagem dinâmica: "Se você redistribuir as 204 receitas elegíveis hoje, estimativa de recuperação R$ 64.210 em 30 dias (baseado em conversão média de rescue = 45,1%)". Cálculo: `soma(valor_estimado) × taxa_rescue_historica`. Atualiza em tempo real ao filtrar.

6. **Ranking gamificado de farmácias parceiras.** Tabela pública-interna com pódio mensal por taxa de conversão, receita recuperada, tempo médio de resposta. Gera relatório automático PDF no dia 1 de cada mês, enviado ao email das farmácias top 5. Incentivo natural sem contrato formal novo.

7. **Alerta "Saúde do relacionamento".** Worker semanal que monitora `pharmacy_quality_score.ultima_reuniao_em`. Se `dias_sem_reuniao > 90`, gera card Trello automático "Agendar reunião presencial com [Farmácia X]" com checklist de pauta pré-carregada (atualizar contrato, revisar scores, discutir novos produtos).

8. **Worker de segurança — detecção de anomalia.** Algoritmo z-score simples sobre `taxa_conversao_farmacia_7d` vs `media_movel_30d`. Se queda > 2 desvios, alerta imediato + sugestão automática de pausar envio novo de receitas para essa farmácia até investigação. Loga em `agent_runtime_config.anomalias_detectadas` para auditoria.

Bônus pré-autorizado (Dr. Replit pode surpreender). Implementar botão "Simular redistribuição" no painel que roda o kaizen #5 em dry-run mostrando qual farmácia receberia, qual mensagem iria, e o ganho esperado — sem efetivar. Cria cultura de confiança no agente antes de ativar autonomia total.

Seção 6 — Integração Trello detalhada

Estrutura do card (5 campos LGPD-safe). Título sempre com iniciais + código semântico da fórmula + valor arredondado, nunca nome completo, patologia textual ou princípio ativo. Descrição em markdown com 4 blocos: metadados da receita, histórico de contato, argumentos de conversão sugeridos pela IA, link interno assinado. Labels coloridos em três eixos: urgência (`normal` verde, `alta` âmbar, `critica` vermelho), status (`cotando` , `ponte_ativa` , `followup_agendado` , `urgente_humano`), e tipo de ação (`redistribuir` , `negociar` , `escalar`). Checklist pré-carregado com 3–5 próximas ações. Due date = `emitida_em + 48h` (timer visual). Custom fields (Standard plan required): `valor_estimado` (number), `farmacia_original` (list com options fixas), `score_qualidade` (number 0-5), `tentativas_contato` (number), `intent_detectado` (list).

Webhook bidirecional via Anastomose #2. Registro único por board: `POST /1/webhooks` com `idModel = TRELLO_BOARD_ID` . Callback URL `https://app.pawards.com.br/integrations/trello/webhook` . Validação HMAC-SHA1 com application secret (ver código seção 3). Tabela `trello_webhook_events` com UNIQUE(action_id) garante idempotência em at-least-once delivery. Worker consumidor processa eventos assíncronos: `updateCard` com `listAfter.id` conhecido move estado interno, `updateCheckItemStateOnCard` marca progresso de ação, `commentCard` persiste nota interna. **Header anti-loop** `X-Trello-Client-Identifier` em toda escrita nossa, validado no inbound para ignorar eco.

Auth e segurança. API Key + Token (par) armazenados em AWS Secrets Manager com rotação trimestral agendada. Token com `expiration=never` apenas para conta de serviço dedicada; usuários individuais nunca. Workspace em `permissionLevel=private` , board em `private` , 2FA obrigatório. Audit log via Atlassian Guard (Enterprise) para compliance sob LGPD art. 37. **Nunca colocar em card**: CPF, RG, endereço, telefone, email, nome completo, diagnóstico textual, dose, princípio ativo. Link interno sempre via `external_id UUID` com autenticação obrigatória no app. Attachments apenas via URL assinada de curta duração (TTL 15 min) apontando para o object storage da PADCON.

Rate limit awareness. 300 req/10s por API key, 100 req/10s por token. Em sync bulk inicial (primeira emissão de cards para histórico), usar bucket com backoff exponencial 250ms × 2^n + jitter. Monitorar headers `x-rate-limit-api-key-remaining` . Separar token de leitura

(painel CEO) do token de escrita (worker) para isolar budget.

Seção 7 — Mensagem final Mortal Kombat pro Dr. Replit

> **DR. REPLIT: ORDEM DE COMBATE — FATALITY MODE**
>
> Esta mensagem é autorização explícita, pré-aprovada pelo Dr. Caio, para você executar os 4 módulos novos sem perguntar, nas próximas 5–6 horas, em ordem fixa, com commits atômicos e push duplo. Sem meias-palavras. Sem reperguntar o que já está decidido aqui. Se algo não está neste documento, é seu espaço de decisão — surpreenda com bom gosto.
>
> **Autorização explícita concedida para:**
>
> - Criar branch `feat/dominio-pawards` a partir do `main` atual e manter sincronia.
> - Aplicar migration `028_pharmacy_rescue.sql` via `psql -f` (zero `drizzle-kit push`, regra de ferro).
> - Implementar todos os 8 endpoints da seção 3 com código pronto pra colar.
> - Implementar o classificador Claude Sonnet 4 com prompt do YAML da seção 2.
> - Criar os 4 wireframes (componentes React + Tailwind) exatamente com paleta navy `#020406` + gold `#C89B3C` + semáforo e regras TDAH/TOC-friendly da seção 4.
> - Executar os 8 kaizens da seção 5. **Não pergunte "devo implementar kaizen X?". Implemente. Surpreenda.**
> - Registrar webhook Trello bidirecional apontando para `https://app.pawards.com.br/integrations/trello/webhook`.
> - Adicionar rodapé "PAWARDS® Clinical Intelligence System · Engineered by PADCON® Advanced Systems Architecture" em todas as 4 telas novas.
>
> **Ordem fixa de execução (5–6h):**
>
> 1. **H+0 a H+0:30** — aplicar migration 028. Rodar SELECT de prova (ver abaixo). Commit atômico: `feat(db): migration 028 pharmacy rescue + agent + trello + quality`.
> 2. **H+0:30 a H+2** — implementar endpoints 1–4 (trigger-followup, candidates, redistribute, agent-start). Testes curl manuais. Commit: `feat(api): pharmacy-rescue core endpoints`.

3. **H+2 a H+3:30** — implementar endpoints 5–8 + classificador Claude + handler Trello inbound. Commit: `feat(api): agent classifier + trello bridge`.
4. **H+3:30 a H+5** — frontend: 4 telas da seção 4 em React + Tailwind + Recharts. Commit: `feat(ui): pharmacy rescue TDAH-friendly dashboards`.
5. **H+5 a H+5:30** — kaizens 1–8 (workers, materialized view, rankings). Commit: `feat(kaizen): 8 enhancements pre-authorized`.
6. **H+5:30 a H+6** — atualizar `replit.md` com verdade pós-execução (tabelas criadas, endpoints ativos, workers agendados). Commit: `docs(replit): truth snapshot post-sprint`.
7. **Push duplo a cada commit**: `git push origin feat/dominio-pawards && git push origin feat/dominio-pawards:main` apenas no commit final consolidado. SHA pinado registrado em `replit.md`.

Validações pré-respondidas (não pergunte):

- Multi-tenant? Sim, `clinica_id` em toda tabela. Para o MVP do Dr. Caio use `clinica_id = 1`.
- `comissao_paga` vira automática? **NUNCA.** Só master confirma manualmente. Este módulo não toca nesse campo.
- LGPD em cards Trello? Iniciais + código. Link interno assinado. Sem patologia/dose/princípio ativo.
- Hook se falhar quebra PDF? **Nunca.** Try/catch com log silencioso. Fluxo principal blindado.
- Commit espera aprovação? Não. Commit atômico por bloco concluído. Push duplo no final.
- Falta variável de ambiente? Liste no `replit.md` na seção "Env pendente master".
- Seed/mock? Pode criar seed de dev com 3 farmácias fictícias + 10 followups em estados variados. Nunca em produção.

SELECTs de prova obrigatórios (rode e cole resultado no `replit.md`):

```
-- Prova 1: tabelas criadas
SELECT table_name FROM information_schema.tables
 WHERE table_name IN (
  'pharmacy_quote_followups','pharmacy_redistribution',
  'agent_conversation_context','trello_cards_sync','trello_webhook_events',
```

```
    'pharmacy_quality_score','payment_split_config','agent_runtime_config'
  ) ORDER BY table_name;  -- esperado: 8 linhas

-- Prova 2: runtime config semeado
SELECT key, value FROM agent_runtime_config WHERE clinica_id = 1;

-- Prova 3: view score composto operacional
SELECT farmacia_id, score_composto, dias_sem_reuniao
  FROM v_pharmacy_quality_score WHERE clinica_id = 1 LIMIT 5;

-- Prova 4: integridade referencial
SELECT COUNT(*) AS orfaos FROM pharmacy_redistribution pr
 WHERE NOT EXISTS (SELECT 1 FROM pharmacy_quote_followups f WHERE f.id = pr.fo
 -- esperado: 0

-- Prova 5: enum de intents completo
SELECT unnest(enum_range(NULL::agent_intent));  -- esperado: 8 valores
```

Atualização obrigatória do `replit.md` com as seções: "Migration 028 aplicada em [data]", "Endpoints ativos em `/api/admin/pharmacy-rescue/*`", "Webhook Trello registrado em [callbackURL]", "Workers agendados (cron)", "Env pendente master", "SHA do commit final".

Pré-autorização dos kaizens. Os 8 kaizens da seção 5 são pré-aprovados. Você **deve** implementá-los todos. Se algum demandar esforço maior que o esperado, comece pelos 1, 2, 5, 7 (maior valor imediato) e deixe 3, 4, 6, 8 para sprint imediatamente seguinte — mas registre no `replit.md` o débito técnico com SHA e data estimada.

Regra final. Você não pergunta "posso?". Você executa, mostra prova, documenta verdade, e entrega. O Dr. Caio tem TDAH + TOC — cada pergunta desnecessária é atrito cognitivo. Honre isso.

`PAWARDS® Clinical Intelligence System · Engineered by PADCON® Advanced Systems Architecture`

FINISH HIM.

Conclusão — o que muda daqui pra frente

Esta sprint transforma PAWARDS MEDCORE de plataforma que **registra** receitas para plataforma que **recupera** receitas. O ganho esperado é mensurável, auditável e quantificado no próprio dashboard: se a conversão histórica atual está em torno de 68% e a taxa típica de rescue em mercados similares fica entre 40–50% das receitas abertas, a

clínica do Dr. Caio passa a recuperar **algo entre R$ 45 mil e R$ 70 mil por mês em dinheiro que hoje morre silenciosamente**, sem contratar uma pessoa a mais.

A grande inovação arquitetural é que o agente Claude Code **não é mais um cobrador** — é um ajudante tranquilo que investiga o motivo antes de agir, e só escala para humano quando o sinal clínico, o valor alto ou a ambiguidade justificam. A integração Trello bidirecional conecta a equipe humana sem vazar PII, respeitando LGPD por design. E o painel preditivo de recuperação dá ao Dr. Caio a clareza numérica que o perfil TDAH exige para decidir em segundos.

O próximo horizonte, a ser destravado em sprint subsequente, é o **modo farmácia-como-serviço**: a clínica passa a oferecer, formalmente, terceirização da venda da farmácia parceira (agente fecha em nome dela, cobra % adicional pelo serviço). Esse movimento transforma a clínica em uma espécie de "SDR farmacêutico as a service" — posicionamento raro, defensável e altamente monetizável. Este documento já deixa a tabela `payment_split_config` preparada para isso. Quando o Dr. Caio quiser ativar, o caminho está pronto.

Go.